MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on Laboratory Work no. 1
Operating Systems
Theme: The OS as a Resource Manager

Done by: st. gr. FAF-242, **Moroșanu Ion**
Checked by: **Reițman Patricia**, *university assistant*
FCIM, UTM

Chișinău – 2026

TABLE OF CONTENTS

INTRODUCTION **3**
Purpose of the work 3
1 FILES AND DIRECTORIES **4**
1.1 The user and the kernel 4
1.2 The file system tree 4
1.3 Creating files and changing their permissions 6
1.4 Answers to the questions 7
2 PROCESSES **8**
2.1 The processes of an idle system 8
2.2 A process in the background 8
2.3 A process seen through /proc 9
2.4 Answers to the questions 10
3 MEMORY **11**
3.1 The memory of the machine and of one process 11
3.2 Answers to the questions 11
4 DEVICES AND STORAGE **13**
4.1 Disks, device files and mounted file systems 13
4.2 Answers to the questions 13
5 THE FOUR RESOURCES **15**
CONCLUSIONS **16**
REFERENCES **17**

INTRODUCTION

Purpose of the work

The purpose of the first laboratory work is to observe the operating system at work as the manager that stands between programs and hardware. Any operating system has four core jobs: it manages files, processes, memory and devices [2]. In this work each of these resources was examined on a running Linux system with standard commands, and the results were recorded together with an explanation of what they show [1].

The following tasks were set for the work:

a) to explore the file system tree, to create, copy, rename and delete files and to change their permissions;

b) to list the running processes, to start a process in the background, to inspect it through the /proc pseudo file system and to terminate it;

c) to read the total and free memory of the system and the memory used by a single process;

d) to list the disks, the mounted file systems and the device files.

All commands were run in the terminal of an Ubuntu 26.04 LTS virtual machine created in Oracle VirtualBox 7.2 on a Windows 11 host. The virtual machine has 4 processors, 4096 MB of memory and a 25 GB virtual disk. The answers to the questions of each part are collected at the end of the chapter of that part, and the file report.md in the folder lab1/ of the repository holds the same answers in short form.

1 FILES AND DIRECTORIES

1.1 The user and the kernel

The first commands identify the user and the kernel (Figure 1.1). The current user is `wancea`, the kernel is Linux `7.0.0-34-generic` for the `x86_64` architecture, and at that moment the system had been running for one minute.

```
wancea@OS-Lab1-2:~$ mkdir -p ~/os-lab1 && cd ~/os-lab1
whoami        # who am I to the OS
uname -a      # which kernel is managing this machine
uptime        # how long has this OS been running
wancea
Linux OS-Lab1-2 7.0.0-34-generic #34-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep  2 14:29
:37 UTC 2026 x86_64 GNU/Linux
 10:51:04 up 1 min,  1 user,  load average: 3.85, 1.48, 0.54
```

Figure 1.1 – Identification of the user, the kernel and the uptime

1.2 The file system tree

The operating system turns the raw disk into a tree of named files and directories, and every file has an owner, a group and a set of permissions. The command `ls -la /` lists the root of this tree (Figure 1.2). All directories directly under / belong to the user root. Two of them show well how the tree is organised: /etc keeps the configuration files of the whole system, such as the settings of NetworkManager or adduser.conf, and /home keeps one personal directory for every user, in this case /home/wancea [5].

An interesting detail in this listing is that /bin, /lib, /lib64 and /sbin are not real directories but symbolic links to usr/bin, usr/lib, usr/lib64 and usr/sbin, so Ubuntu keeps all programs under /usr. The directories /proc and /sys have a size of 0 because they are not stored on the disk: the kernel generates their content at the moment it is read. The directory /tmp has the permissions `drwxrwxrwt`, where the final t is the sticky bit, which lets every user create files there but delete only their own.

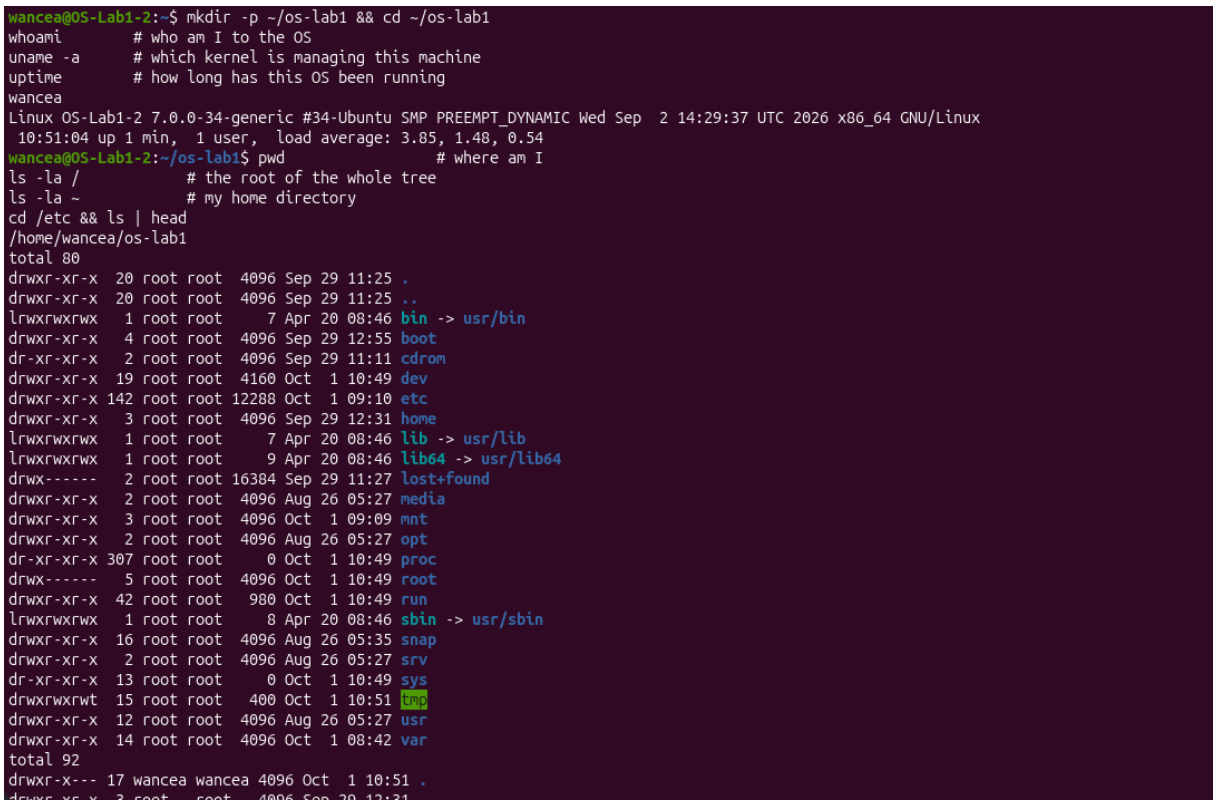


Figure 1.2 – The root of the file system tree

The home directory (Figure 1.3) has the permissions `drwxr-x---`, so other users cannot enter it. Besides the usual folders such as Desktop and Documents, it contains hidden configuration files whose names start with a dot, for example .bashrc and .profile.

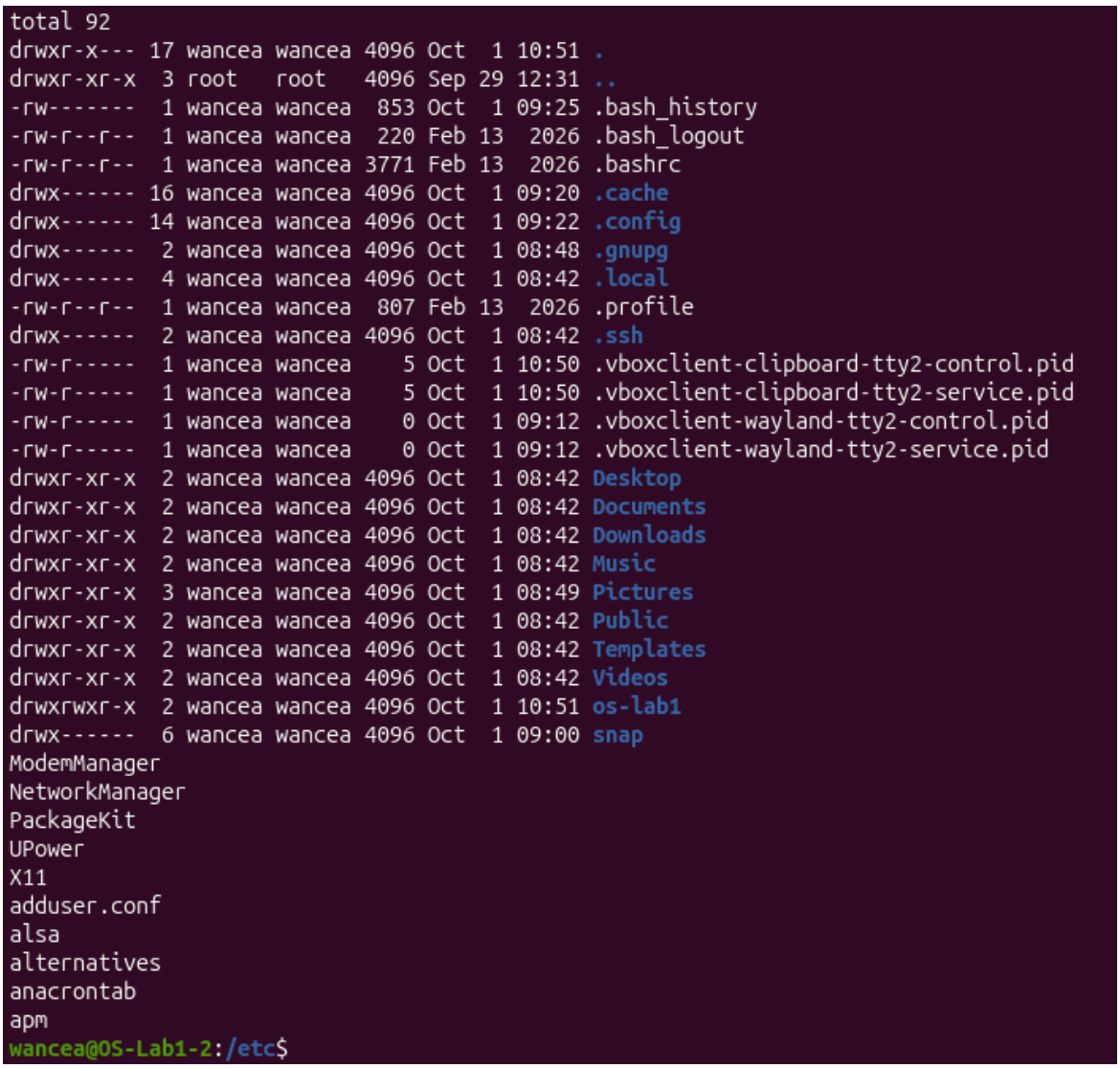


Figure 1.3 – The home directory and the first entries of /etc

1.3 Creating files and changing their permissions

In the folder ~/os-lab1/demo the file note.txt was created with `echo`, copied to copy.txt, renamed to renamed.txt with `mv` and then removed with `rm` (Figure 1.4). Both files have a size of 24 bytes, which is the length of the text "hello operating systems" plus the end of line character. They are owned by the user wancea and by the group wancea, because a new file takes the owner and the primary group of the process that creates it, and on Ubuntu every user has a private group with the same name.

The first ten characters printed by `ls -l` describe the type and the permissions of a file. Before the change note.txt had `-rw-rw-r--`, so the owner and the group could read and write it and the other users could only read it. This default comes from the umask value 0002. After `chmod 600` the same line shows `-rw-------`. The first character means a regular file (a directory would show d). The next three characters, rw-, are the rights of the owner: read and write, but not execute. The last two groups

of three are empty, so the group and the other users have no access at all. The command `chmod 644` then gave read access back to the group and to the other users.

```
wancea@OS-Lab1-2:/etc$ cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
wancea@OS-Lab1-2:~/os-lab1/demo$ mv copy.txt renamed.txt
ls -l
rm renamed.txt
total 8
-rw-rw-r-- 1 wancea wancea 24 Oct  1 10:53 note.txt
-rw-rw-r-- 1 wancea wancea 24 Oct  1 10:53 renamed.txt
wancea@OS-Lab1-2:~/os-lab1/demo$ ls -l note.txt      # read the rwx / owner / group columns
chmod 600 note.txt  # owner-only read+write
ls -l note.txt
chmod 644 note.txt
-rw-rw-r-- 1 wancea wancea 24 Oct  1 10:53 note.txt
-rw------- 1 wancea wancea 24 Oct  1 10:53 note.txt
wancea@OS-Lab1-2:~/os-lab1/demo$
```

Figure 1.4 – File operations and permission changes in ~/os-lab1/demo

1.4 Answers to the questions

1. The files created in ~/os-lab1/demo belong to the user wancea and to the group wancea, the private group that Ubuntu makes for every user (Figure 1.4).

2. After `chmod 600` the line of note.txt starts with `-rw-------`. The first character, -, means a regular file. The next three, rw-, give the owner the right to read and write but not to execute. The last two groups, --- and ---, give the group and all the other users no rights at all.

3. /etc holds the configuration files of the whole system, for example the settings of NetworkManager. /home holds one personal folder for every user, here /home/wancea.

2 PROCESSES

2.1 The processes of an idle system

Every running program is a process that the kernel created, isolates from the others and schedules on the processor. The live view of `top` (Figure 2.1) showed 245 tasks on the idle virtual machine: 1 running, 244 sleeping, 0 stopped and 0 zombie, with the processor idle 89.2% of the time. The programs that used the processor most were the terminal emulator ptyxis and gnome-shell, the graphical desktop, which also held the most memory, about 412 MiB of resident memory (12.2% of the RAM).

The process with PID 1 is systemd, owned by root. It is the first process that the kernel starts after boot (the init process). It starts and supervises the system services and adopts every process whose parent has ended, so all user processes descend from it [3]. The process with PID 2 is kthreadd, the kernel thread that creates the other kernel threads, such as the many kworker entries in the list. They have a virtual size of 0 because they run inside the kernel and have no user memory of their own.

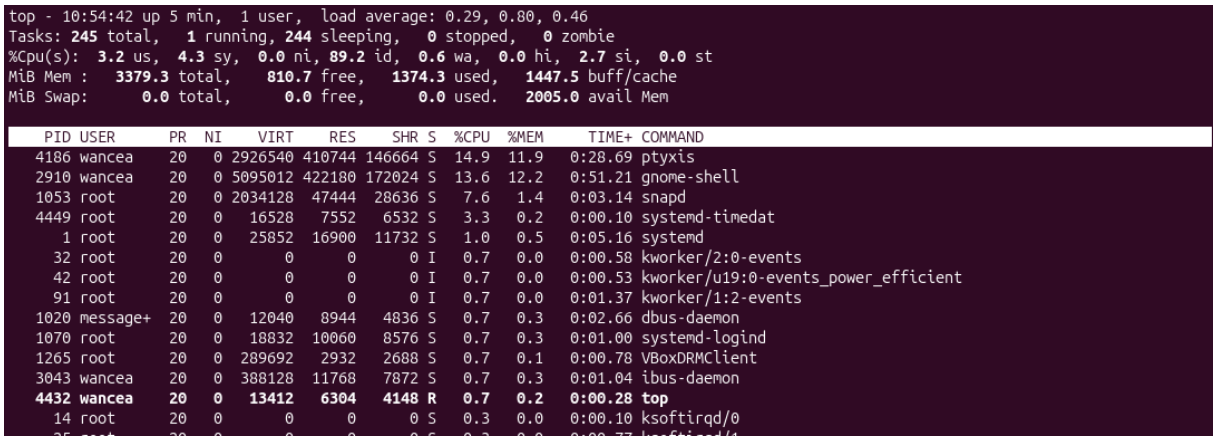


Figure 2.1 – The live process view of top

2.2 A process in the background

A process was then started in the background with `sleep 300 &`. The shell printed `[1] 4485`, which is the job number followed by the PID (Figure 2.2). The command `jobs` showed it as Running, and `ps -ef | grep sleep` found it with the parent PID 4228, which is the bash shell that started it. The line of grep appears as well because grep itself was a running process at that moment. The first attempt to stop the process failed: the command `kill <PID>` was typed literally, and bash read the character < as an input redirection, so it reported a syntax error. With the real number, `kill 4485` asked the OS to terminate the process, and `jobs` then reported it as Terminated.

```
wancea@OS-Lab1-2:~/os-lab1/demo$ sleep 300 &        # run in the background; note the PID printed
jobs                # shell view of background jobs
ps -ef | grep sleep # find it among all processes
kill <PID>          # ask the OS to terminate it (use the PID above)
[1] 4485
[1]+  Running                 sleep 300 &
wancea      4485    4228  0 10:54 pts/0    00:00:00 sleep 300
wancea      4487    4228  0 10:54 pts/0    00:00:00 grep --color=auto sleep
bash: syntax error near unexpected token `newline'
wancea@OS-Lab1-2:~/os-lab1/demo$ ^[[200~kill 4485

kill: command not found
wancea@OS-Lab1-2:~/os-lab1/demo$ jobs~
jobs~: command not found
wancea@OS-Lab1-2:~/os-lab1/demo$ kill 4485
jobs
[1]+  Running                 sleep 300 &
[1]+  Terminated              sleep 300
wancea@OS-Lab1-2:~/os-lab1/demo$
```

Figure 2.2 – Starting, finding and terminating a background process

2.3 A process seen through /proc

The kernel also shows every process as a directory in the /proc pseudo file system [4]. For a new sleep process with PID 5550, the directory /proc/5550 contained entries such as cmdline, cwd, environ, exe, fd, maps and status (Figure 2.3). The status file reported `State: S (sleeping)`, because sleep only waits for a timer and does not use the processor. The same file shows the parent PID 4228, the user and group id 1000 of wancea and a virtual memory size of 16112 kB.

```
wancea@OS-Lab1-2:~/os-lab1/demo$ kill $PID
[1]+  Terminated              sleep 300
wancea@OS-Lab1-2:~/os-lab1/demo$ ~sleep: command not found
^C
wancea@OS-Lab1-2:~/os-lab1/demo$ sleep 300 &
[1] 5550
wancea@OS-Lab1-2:~/os-lab1/demo$ PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20   # state, memory, threads
kill $PID
arch_status  comm               fdinfo            loginuid    net           patch_state  setgroups     syscall
attr         coredump_filter    gid_map           map_files   ns            personality  smaps         task
autogroup    cpu_resctrl_groups io                maps        numa_maps     projid_map   smaps_rollup  timens_offsets
auxv         cwd                ksm_merging_pages mem         oom_adj       root         stack         timers
cgroup       environ            ksm_stat          mountinfo   oom_score     sched        stat          timerslack_ns
clear_refs   exe                latency           mounts      oom_score_adj schedstat    statm         uid_map
cmdline      fd                 limits            mountstats  pagemap       sessionid    status        wchan
Name:   sleep
Umask:  0002
State:  S (sleeping)
Tgid:   5550
Ngid:   0
Pid:    5550
PPid:   4228
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 4 24 27 30 46 100 111 114 1000
NStgid: 5550
NSpid:  5550
NSpgid: 5550
NSsid:  4228
Kthread:        0
VmPeak:    16112 kB
VmSize:    16112 kB
VmLck:         0 kB
[1]+  Terminated              sleep 300
wancea@OS-Lab1-2:~/os-lab1/demo$
```

Figure 2.3 – The /proc directory of a process and its status file

2.4 Answers to the questions

1. The process with PID 1 is systemd. It is the init process, the first program that the kernel starts after boot; it starts the system services and adopts every process whose parent has ended.

2. About 245 processes were running on the idle virtual machine: top reported 245 tasks, 1 running and 244 sleeping (Figure 2.1).

3. The line `State:` in /proc/5550/status said `S (sleeping)`: the process only waits for its timer and does not use the processor (Figure 2.3).

3 MEMORY

3.1 The memory of the machine and of one process

The commands `free -h` and `cat /proc/meminfo` give the memory totals of the whole machine (Figure 3.1). The virtual machine has 3.3 GiB of RAM in total (MemTotal 3460392 kB), a little less than the 4096 MB set in VirtualBox, because the memory reserved by the firmware and by the kernel at boot is not counted. At the moment of the command only 348 MiB were completely free, while 1.9 GiB were available. The difference comes from the 1.8 GiB shown as buff/cache: Linux uses RAM that no program needs as a cache for disk data and gives it back as soon as a program asks for memory, so a small free value is normal on Linux.

Swap is an area on the disk to which the kernel can move memory pages that have not been used for a while when the RAM is full. It lets the system run more programs than fit in physical memory, at the cost of speed, since the disk is much slower than RAM. On this virtual machine no swap is configured: both `free` and `top` show 0 B of swap.

The resident memory of a bare sleep process, read from the VmRSS line of its status file, was only 224 kB, although the same program has a virtual size of about 16 MB. This was a small surprise at first. The explanation is demand paging: the kernel places a page in RAM only when the process actually uses it, and sleep uses very little before it starts waiting. Most of the 16 MB is address space reserved for the program and its libraries, not memory that is really occupied.

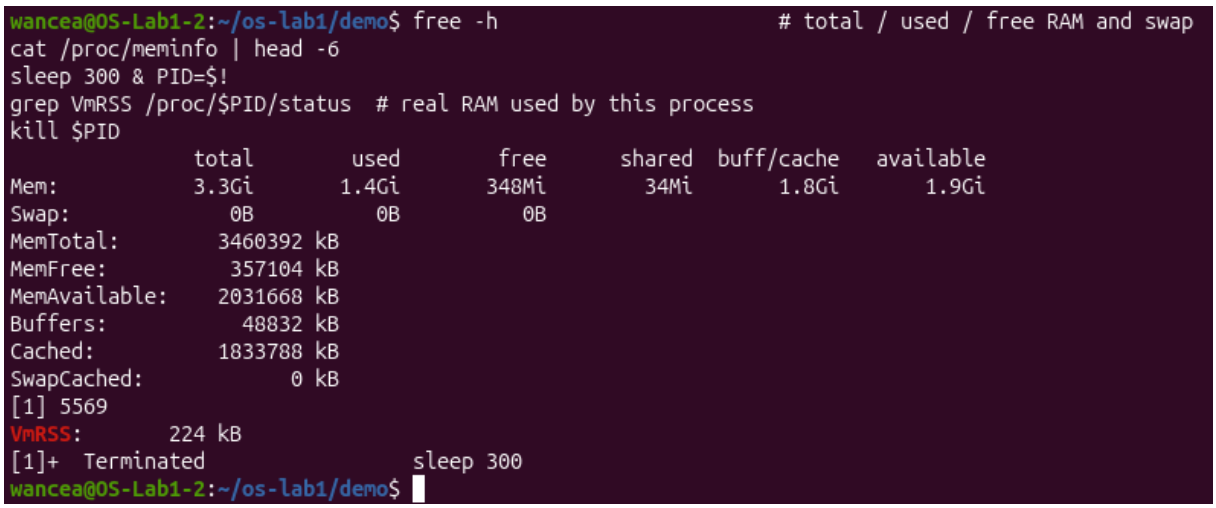


Figure 3.1 – Memory totals and the resident memory of one process

3.2 Answers to the questions

1. The virtual machine has 3.3 GiB of RAM in total (MemTotal 3460392 kB). 348 MiB were completely free, and 1.9 GiB were available, because the 1.8 GiB of disk cache can be given back at once.

2. Swap is space on the disk where the kernel moves memory pages that were not used for a while when the RAM is full. No swap is configured on this machine: free shows 0B.

3. A bare sleep process uses only 224 kB of resident memory (VmRSS). It was a surprise, since its virtual size is about 16 MB, but the kernel loads a page into RAM only when the process touches it, and sleep touches very little.

4 DEVICES AND STORAGE

4.1 Disks, device files and mounted file systems

The operating system hides every disk and device behind a uniform interface that looks like a file. The command `lsblk` lists the block devices (Figure 4.1). The virtual disk sda has 25 GB and two partitions: sda1 (1 GB) mounted on /boot/efi and sda2 (23.9 GB) mounted on /. The output of `mount` confirms that the root file system / is on the device /dev/sda2 and uses the ext4 format. The device sr0 is the virtual optical drive, which still had the VirtualBox Guest Additions disc inserted. The many loop devices are not real disks: each of them is the compressed image of a snap package, for example Firefox, mounted as a read only file system. The command `du -sh ~/os-lab1` reported 12K for the work folder.

In /dev every device appears as a file. For example, /dev/console is a character device (the line starts with c, major number 5, minor number 1) that stands for the system console, and /dev/cdrom is a link to sr0, the CD drive of the virtual machine. The output of `mount` also shows that /dev and /sys are themselves file systems of the types devtmpfs and sysfs that the kernel fills in memory.

```
loop3    7:3    0  16.5M  1 loop /snap/firmware-updater/226
loop4    7:4    0 260.3M  1 loop /snap/firefox/8763
loop5    7:5    0 614.5M  1 loop /snap/gnome-46-2404/164
loop6    7:6    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop7    7:7    0   402M  1 loop /snap/mesa-2404/1839
loop8    7:8    0   1.5M  1 loop /snap/hwctl/123
loop9    7:9    0  11.8M  1 loop /snap/snap-store/1390
loop10   7:10   0  18.8M  1 loop /snap/prompting-client/222
loop11   7:11   0  50.1M  1 loop /snap/snapd/27710
loop12   7:12   0   828K  1 loop /snap/snapd-desktop-integration/391
loop13   7:13   0  66.8M  1 loop /snap/core24/2124
loop14   7:14   0  18.4M  1 loop /snap/prompting-client/228
loop15   7:15   0  14.2M  1 loop /snap/desktop-security-center/188
sda      8:0    0    25G  0 disk
├─sda1   8:1    0     1G  0 part /boot/efi
└─sda2   8:2    0  23.9G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/wancea/VBox_GAs_7.2.20
12K     /home/wancea/os-lab1
total 0
crw-r--r--  1 root   root     10, 235 Oct  1 10:49 autofs
drwxr-xr-x  2 root   root        440 Oct  1 10:55 block
drwxr-xr-x  2 root   root         80 Oct  1 10:49 bsg
crw-------  1 root   root     10, 234 Oct  1 10:49 btrfs-control
drwxr-xr-x  3 root   root         60 Oct  1 10:49 bus
lrwxrwxrwx+ 1 root   root          3 Oct  1 10:49 cdrom -> sr0
drwxr-xr-x  2 root   root       3780 Oct  1 10:50 char
crw-------  1 root   tty       5,   1 Oct  1 10:49 console
lrwxrwxrwx  1 root   root         11 Oct  1 10:49 core -> /proc/kcore
tmpfs on /run type tmpfs (rw,nosuid,nodev,size=692080k,nr_inodes=819200,mode=755,inode64)
/dev/sda2 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=756856k,nr_inodes=189214,mode=755,inode64)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,usrquota)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot,memory_hugetlb_accounting)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
efivarfs on /sys/firmware/efi/efivars type efivarfs (rw,nosuid,nodev,noexec,relatime)
wancea@OS-Lab1-2:~/os-lab1/demo$
```

Figure 4.1 – Block devices, device files and mounted file systems

4.2 Answers to the questions

1. The root file system / is mounted on the device /dev/sda2, the second partition of the virtual disk sda, and it uses the ext4 format.

2. /dev/console is a character device (major number 5, minor number 1) that stands for the system console, the terminal to which the kernel writes its messages.

3. "Everything is a file" means that disks, terminals, kernel information and even running processes are reached through names in the same tree and are read or written with the same operations as ordinary files.

5 THE FOUR RESOURCES

Table 5.1 lists the four resources that the operating system manages and the command that made each of them visible in this work.

Table 5.1 – Resources managed by the operating system

Resource	What the operating system does	Command used
Files	organises the disk into a tree of named files with owners and permissions	`ls -l`
Processes	creates, schedules and terminates the running programs	`ps -ef`, `top`
Memory	shares the physical RAM between the processes	`free -h`
Devices	presents disks and other hardware as files	`lsblk`

The operating system manages four resources: files, processes, memory and devices. The file tree and its permissions were visible with `ls -l`, the processes with `ps -ef` and `top`, the memory with `free -h` and the devices with `lsblk`. Each of these commands only reads information that the kernel already keeps about the resource, which shows that the operating system is the one that tracks and shares everything that programs use.

CONCLUSIONS

The work showed the four core jobs of an operating system on a running Ubuntu system. The file system tree, the permission bits and the owner of every file showed how the operating system controls the access to data. The commands `top` and `ps` and the /proc directory showed that even an idle desktop runs about 245 processes and that the kernel keeps detailed information about each of them, such as its state, its parent and its memory. The output of `free` and /proc/meminfo showed that Linux keeps unused memory busy as a disk cache and that a small program like sleep holds only a few hundred kilobytes of real RAM. Finally, `lsblk`, `mount` and the /dev directory showed that disks and other devices are reached through files.

A practical lesson came from a mistake: a value written in angle brackets in the instructions, such as <PID>, has to be replaced with the real number, because the shell gives a special meaning to characters like < and >. The work was also done inside a virtual machine, so the hardware that Linux reported, such as the disk sda and the drive sr0, was emulated by VirtualBox, which is itself a program managed by the Windows host. In this way two operating systems were managing resources at the same time.

REFERENCES

1. The OS as a Resource Manager. Laboratory Work 1, Operating Systems course. Faculty of Computers, Informatics and Microelectronics, Technical University of Moldova, 2026.
2. SILBERSCHATZ, A., GALVIN, P. B., GAGNE, G. Operating System Concepts. 10th edition. Hoboken: Wiley, 2018.
3. systemd(1). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man1/systemd.1.html
4. proc(5). Linux manual page [online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html
5. Filesystem Hierarchy Standard, version 3.0. The Linux Foundation, 2015 [online]. Available: https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.html